

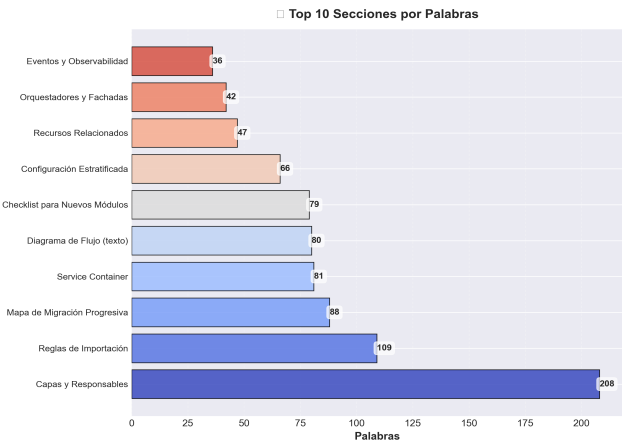
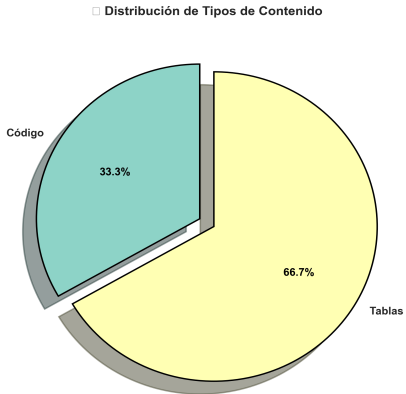
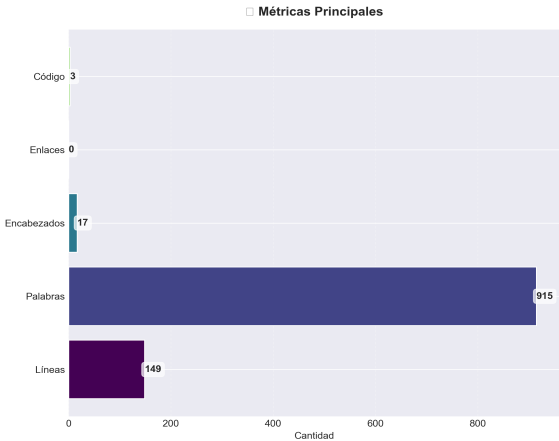
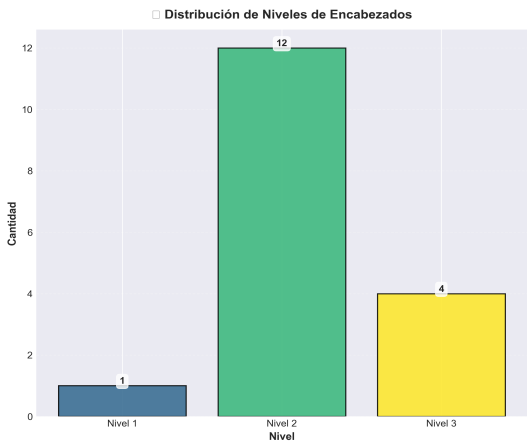
Layers

Análisis y Documentación Completa
Generado el: 25 de November de 2025 a las 11:59

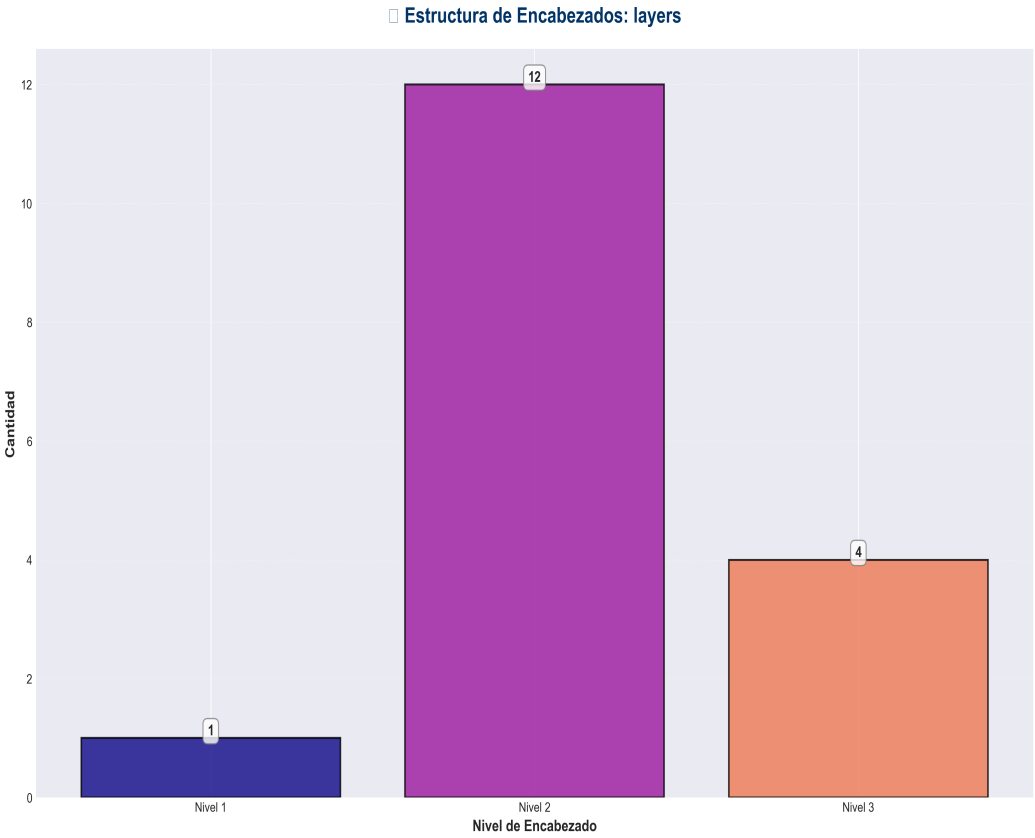
Análisis Visual Completo

Dashboard Principal

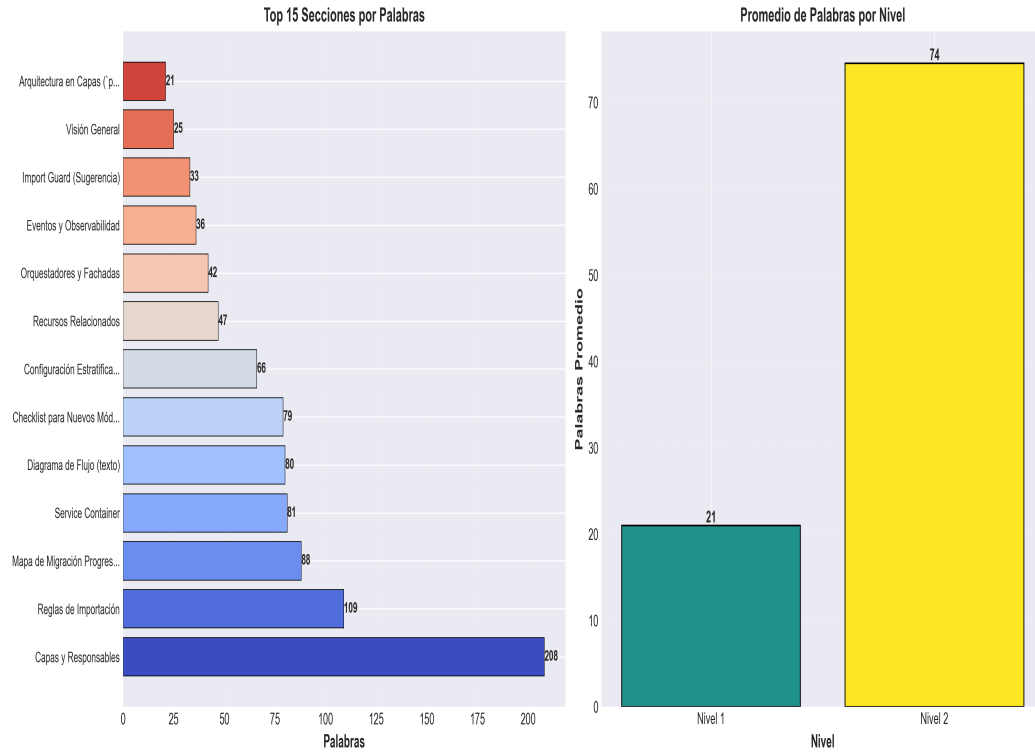
Análisis Completo: layers



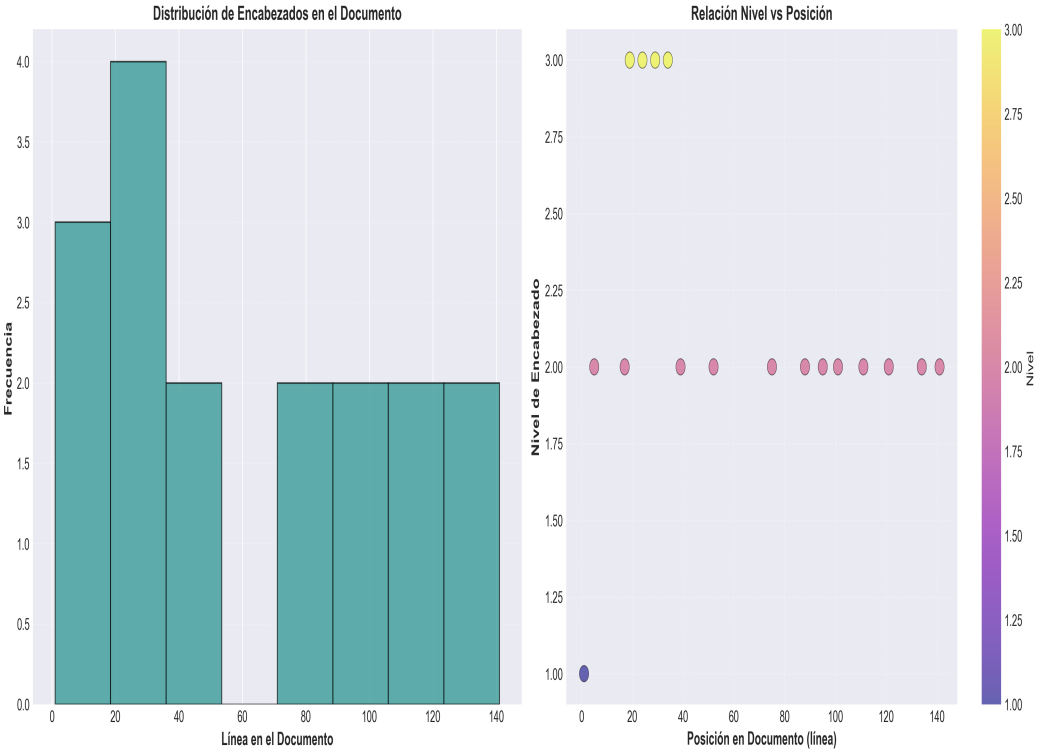
Análisis Adicionales



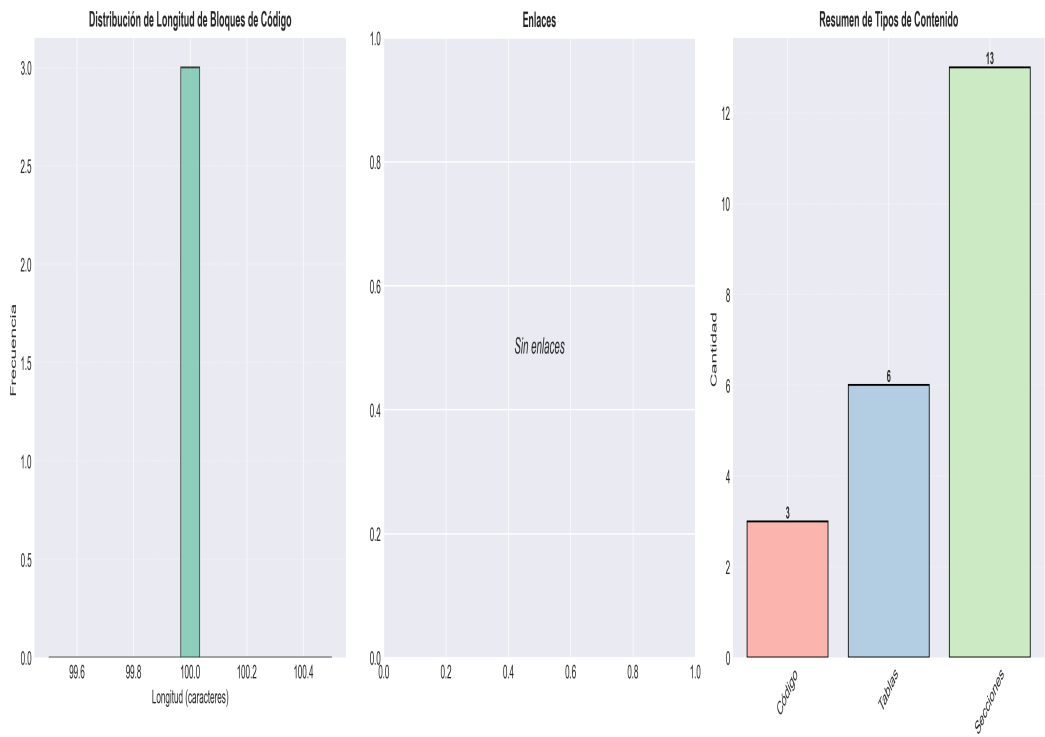
□ Análisis de Complejidad: layers



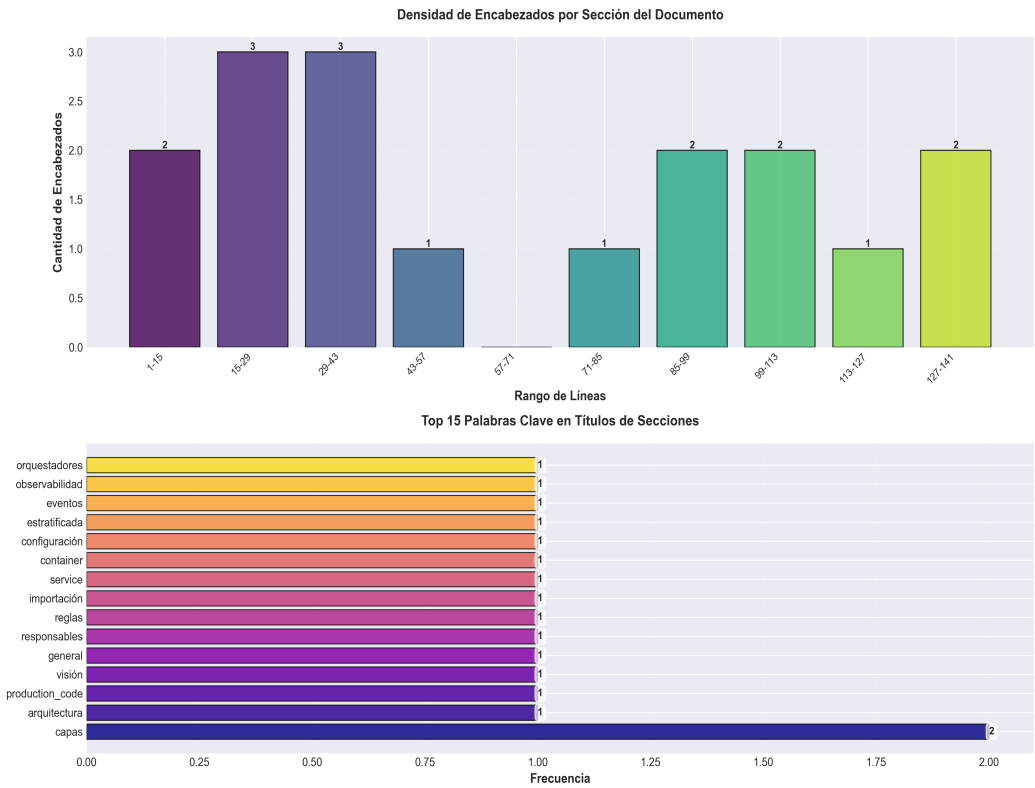
□ Análisis Temporal y Distribución: layers



■ Análisis Detallado de Contenido: layers



□ **Análisis de Densidad y Frecuencia: layers**



Resumen Ejecutivo: Este documento contiene 915 palabras distribuidas en 149 líneas, con 17 encabezados organizados en 13 secciones principales.

Estadísticas del Documento

Métrica	Valor
Total de líneas	149
Total de palabras	915
Total de caracteres	8,148
Encabezados	17
Bloques de código	3
Tablas	6
Enlaces	0
Imágenes	0
Secciones principales	13

Índice de Secciones

Nivel	Título	Línea
1	Arquitectura en Capas (`production_code`)	1
2	Visión General	5
2	Capas y Responsables	17
2	Reglas de Importación	39
2	Service Container	52
2	Configuración Estratificada	75
2	Eventos y Observabilidad	88
2	Orquestadores y Fachadas	95
2	Mapa de Migración Progresiva	101
2	Checklist para Nuevos Módulos	111
2	Diagrama de Flujo (texto)	121
2	Import Guard (Sugerencia)	134
2	Recursos Relacionados	141

Resumen del Contenido

Arquitectura en Capas (`production\_code`)

> Objetivo: aislar responsabilidades, minimizar dependencias circulares y permitir intercambio rápido de módulos/modelos manteniendo pipelines estables.

Visión General

Presentación (FastAPI/CLI/UI/WebSocket)

↓ (DTOs + Validaciones)

Aplicación (Servicios/Orquestadores)

↓ (Contratos + Eventos)

Dominio (Entidades/Reglas/Contratos)

↓ (Adapters registrados)

Infraestructura (Providers/Drivers externos)

Capas y Responsables

Presentación

- Archivos: `api\_unified.py`, `api\_server.py`, `chat\_server.py`, `cli.py`, `cli\_unified.py`, `dashboard.html`, `api\_middleware.py`, `api\_auth.py`.

- Funciones: exponer HTTP/WebSocket/CLI, validar inputs con Pydantic, mapear DTOs ↔ modelos de dominio, aplicar middleware (auth, rate limiting, logging) sin acceso directo a `core/` o `memory/`.

- Dependencias permitidas: `application.service\_container`, contratos DTO propios y utilidades específicas de presentación (ej. `api\_utils` limitado a serialización/logging).

Aplicación

- Archivos: `services/`, `integration\_pipeline.py`, `docs\_generator.py`, `monitoring\_system.py`, `testing\_suite.py`, `improve\_models.py`, coordinadores de `multimodal\_api/`, `sora/`.

- Funciones: implementar casos de uso, coordinar múltiples módulos de dominio, aplicar políticas (reintentos con ``safe_execute``, cuotas, scheduling), emitir eventos a monitoreo/auditoría.
- Depende exclusivamente de contratos definidos en Dominio y de factories/providers expuestos por Infraestructura a través del ``ServiceContainer``.

Dominio

- Archivos: ``core/``, ``memory/``, ``research/``, ``inference/``, ``architecture/``, ``techniques/``, ``code/``, ``best/``, ``redundancy/``, ``model_data/``.
- Funciones: entidades, agregados, validaciones profundas, métricas internas y contratos (Protocol/ABC) como ``MemoryModule``, ``ChatEngine``, ``RedundancyModule``, ``ConfigRepository``.
- No conoce frameworks externos; cualquier necesidad se expresa como contrato/evento.

Infraestructura

- Archivos: ``api_utils.py``, ``config_manager.py``, adaptadores concretos en ``services/``, ``multimodal_api/``, ``sora/``, ``model_data/``, ``static/``, integración con caches, storage, tracing.
- Funciones: implementar contratos de Dominio usando dependencias reales (FastAPI, httpx/requests, Ray, wandb, Redis, S3, Prometheus), gestionar configuración multi-formato y exponer factories/registries para DI.
- Se agrupa en ``infrastructure/providers/`` (LLM, memoria, redundancia, monitoreo, storage).

Reglas de Importación

| Puede importar → | Presentación | Aplicación | Dominio |
Infraestructura |

|-----|-----|-----|-----|-----|

| ****Presentación**** | — | ■ | ■ | Solo helpers propios (``api_utils``) |

| ****Aplicación**** | ■ | — | ■ (contratos) | ■ (providers vía
container/factories) |

| ****Dominio**** | ■ | ■ | — | ■ |

| ****Infraestructura**** | ■ | ■ (para exponer providers/fachadas) | ■
(implementa contratos) | — |

Notas:

- ``services/`` actúa como capa de aplicación ****si**** expone métodos de orquestación; los adaptadores concretos que tocan SDKs externos deben moverse a ``infrastructure/providers``.
- Cualquier importación prohibida debe reemplazarse con una interfaz/contrato en ``domain/contracts``.

Service Container

Archivo sugerido: ``application/service_container.py``.

Responsabilidades:

1. Construir configuraciones estratificadas (``PresentationConfig``, ``ApplicationConfig``, ``DomainConfig``, ``InfrastructureConfig``) utilizando ``ConfigManager``.
2. Registrar providers (memoria, redundancia, chat engines, telemetría) por nombre.
3. Exponer factories ``get_pipeline_service()``, ``get_chat_service()``, etc., utilizados por ``Depends`` en FastAPI, CLI y pruebas.
4. Gestionar ciclo de vida (singletons vs scoped) y recursos compartidos (clientes httpx, pools).

Ejemplo de integración en FastAPI:

```
container = ServiceContainer.from_env()
app = FastAPI(...)
app.state.container = container

@router.post("/pipeline/run")
def run_pipeline(dto: PipelineDTO, service: PipelineService = Depends(container.provide_pipeline)):
    return service.run(dto.to_command())
```

Configuración Estratificada

``config_manager.py`` debe exponer:

- ``PresentationConfig``: flags de middlewares, CORS, rate limiting.
- ``ApplicationConfig``: cuotas, políticas de reintento (``safe_execute``), scheduling.
- ``DomainConfig``: hiperparámetros, límites de memoria, thresholds.
- ``InfrastructureConfig``: credenciales externas, endpoints, tipos de provider habilitados.

Validaciones cruzadas:

- Si ``enable_chat=True`` $\Rightarrow$ ``InfrastructureConfig.chat.provider`` debe existir.
- Si ``redundancy.mode=ensemble`` $\Rightarrow$ deben declararse al menos dos ``RedundancyModule``.
- Observabilidad habilitada $\Rightarrow$ ``MonitoringProvider`` configurado (Prometheus, W&B;, etc.).

Eventos y Observabilidad

Dominio emite eventos ligeros (``MessageProcessed``, ``MemoryThresholdReached``, ``PipelineStepFailed``). Aplicación los mapea a métricas/logs y decide la severidad. Infraestructura provee los sinks:

- ``MonitoringProvider`` $\rightarrow$ Prometheus/OpenTelemetry.
- ``AuditProvider`` $\rightarrow$ Kafka/S3/DB.
- ``NotificationProvider`` $\rightarrow$ Slack/Email.

Orquestadores y Fachadas

- Consolidar orquestadores en ``application/orchestrators/`` (``DocsOrchestrator``, ``MonitoringOrchestrator``, ``MultimodalOrchestrator``).
- Cada façade (``MemoryService``, ``RedundancyService``, ``ChatService``) encapsula interacción con módulos de dominio y se usa desde APIs/CLI.
- Reglas: un orquestador solo coordina servicios/fachadas, nunca importa implementaciones concretas del dominio.

[Nota: El documento original tiene 149 líneas. Se muestra un resumen de las primeras 100 líneas.]